



Distributed File Storage

High-Performance Fault-Tolerant Object Cluster

GO
1.21

C++17

REED-SOLOMON
(3+1)

GRPC
V1.62

SYSTEM ARCHITECTURE & TECHNICAL MANUAL

Distributed File Storage (DFS) Cluster

A resilient, distributed object storage system utilizing Reed-Solomon Erasure Coding across a 4-node storage fabric, coordinated by Go with C++ high-throughput I/O storage daemons.

3 + 1

Data & Parity Shards
(Galois Field $GF(2^8)$)

25%

Cluster Loss
Tolerance (1 of 4
nodes offline)

SHA-256

Per-shard
cryptographic
integrity validation

gRPC

Zero-copy binary
inter-node
communication

Author: Engineering Team • Distributed Systems Architecture

Target Release: v1.0.0 Production Spec • September 2026

1. Executive Summary & Table of Contents

TABLE OF CONTENTS

1. Executive Summary & Table of Contents	Page 2
2. High-Level Architecture & Cluster Topology	Page 3
3. Reed-Solomon Erasure Coding Mathematics & Workflow	Page 4
4. Subsystem Specifications (Coordinator & Storage Nodes)	Page 5
5. Inter-Service Protocol & API Reference (REST + gRPC)	Page 6
6. Fault Tolerance, Failure Scenarios & Verification	Page 7
7. Deployment, Configuration & Operations Guide	Page 8

Executive Overview

The **Distributed File Storage (DFS)** platform is an industrial-grade, fault-tolerant object store engineered to provide high data durability without the 200–300% storage overhead typical of naive data replication. By implementing **Reed-Solomon (3+1) Erasure Coding**, DFS reduces storage overhead to just **33.3%** while guaranteeing total recoverability against single-node complete failure.

The architecture combines two specialized computing tiers:

- **Go Coordinator Tier:** Handles ingress HTTP traffic, in-memory object cataloging, concurrent Reed-Solomon matrix encoding/decoding, and real-time node health heartbeat polling via gRPC.
- **C++17 Storage Node Tier:** Four high-performance standalone storage daemons leveraging direct POSIX disk I/O, SHA-256 verification per chunk, and lightweight thread-safe atomic data operations.
- **Browser Operations Console:** A zero-dependency single-page application providing real-time telemetry, live storage capacity visualization, dynamic shard flow animation, and interactive node kill/recovery triggers.

Core Reliability Guarantee

Any file stored on the cluster is partitioned into 3 data shards and 1 parity shard. Any 3 of the 4 shards are mathematically proven to reconstruct the 100% original binary stream with zero data loss.

2. High-Level Architecture & Cluster Topology

The DFS cluster employs a decoupled orchestrator-worker topology. Clients interact exclusively with the Coordinator via standard HTTP REST APIs, completely isolating internal node topology and network addressing from end users.

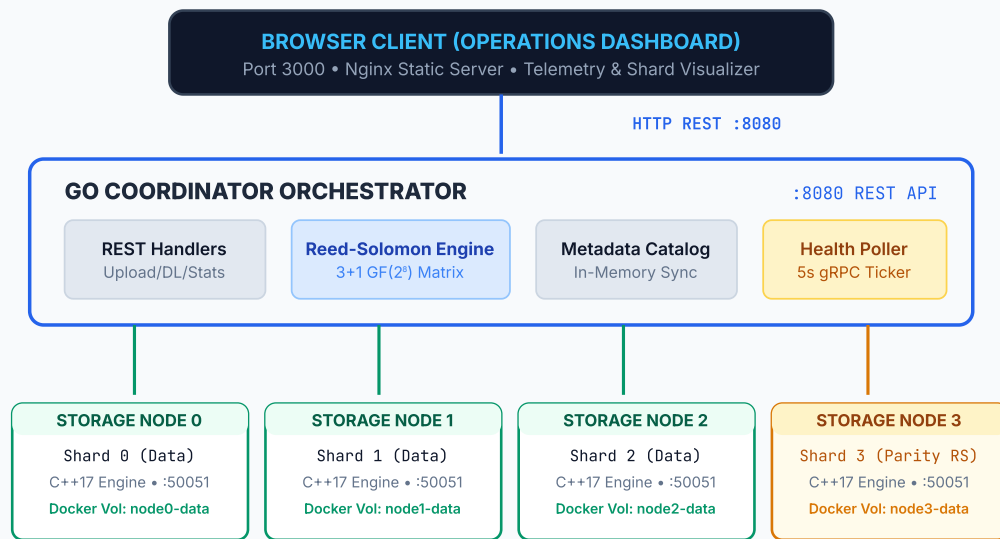


Figure 2.1: Distributed File Storage 3-tier logical topology and inter-service boundaries.

Topology Highlights

Decoupled Control & Data Planes

The coordinator operates as the single source of truth for routing, metadata, and erasure coding. Individual storage nodes have no knowledge of other nodes or overall file structures, dramatically simplifying storage node design.

Zero-Shared Storage (Shared-Nothing)

Each C++ storage container binds an isolated Docker volume (``node0-data`` through ``node3-data``). There is no distributed lock manager (DLM) or shared file system overhead, ensuring maximum parallel disk throughput.

3. Reed-Solomon Erasure Coding Mathematics & Workflow

Mathematical Foundations

Traditional 3x replication triples the storage footprint (300% overhead). DFS implements **Reed-Solomon (3+1)** erasure coding over a **Galois Field $GF(2^8)$** , reducing parity overhead to just **33.3%**:

Shard 0: Data Block 1

Bytes `0 .. N/3` of original file stream. Stored on Node 0.

Shard 1: Data Block 2

Bytes `N/3 .. 2N/3` of original file stream. Stored on Node 1.

Shard 2: Data Block 3

Bytes `2N/3 .. N` with zero padding. Stored on Node 2.

Shard 3: Parity Block

Linear combination over $GF(2^8)$: `P = G · [D0, D1, D2]T`. Stored on Node 3.

Encoding and Striping Pipeline

- Ingress & Sizing:** Inbound file bytes are received via HTTP multipart upload. Length is verified.
- Padding & Alignment:** The payload is padded with zeros until its size is an exact multiple of `3` bytes (Data Shards `k = 3`).
- Matrix Multiplication:** Using the Vandermonde-derived generator matrix, parity shard 3 is synthesized:

$$\begin{bmatrix} D0 \\ D1 \\ D2 \\ P0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ g0 & g1 & g2 \end{bmatrix} \cdot \begin{bmatrix} D0 \\ D1 \\ D2 \end{bmatrix}$$

- Checksum Generation:** SHA-256 hashes are calculated for each of the 4 generated shards.
- Parallel Dispatch:** Shards are concurrently transmitted via gRPC `StoreShard` requests to their respective nodes.

Reconstruction on Node Failure

When an object download is requested and one of the data nodes is offline or reports a missing shard:

- Healthy Fast-Path (4/4 Nodes Up):** Shards 0, 1, and 2 are fetched in parallel. The coordinator concatenates them directly and truncates padding bytes. Parity shard 3 is not requested, minimizing network bandwidth.
- Degraded Recovery Path (1 Node Down):** If Node 1 fails, the coordinator gathers Shard 0, Shard 2, and Parity Shard 3. By inverting the corresponding 3x3 submatrix in Galois Field $GF(2^8)$, Shard 1 is mathematically reconstituted in memory without needing to query the failed node.

Fault-Tolerance Boundary

Because $k = 3$ and $m = 1$, the cluster strictly tolerates **any 1 node failure**. If 2 nodes fail simultaneously, insufficient equations remain in the linear system and recovery is mathematically impossible.

4. Subsystem Specifications

Coordinator Subsystem (Go)

The coordinator is built in Go 1.21 using only standard library HTTP routing combined with the official gRPC-Go runtime.

PACKAGE / MODULE	LOCATION	RESPONSIBILITY
<code>main</code>	<code>coordinator/main.go</code>	Service wiring, environment loading, graceful shutdown signal hooks.
<code>api</code>	<code>coordinator/api/handler.go</code>	HTTP REST endpoints for upload, streaming download, cluster status, and health.
<code>distributor</code>	<code>coordinator/distributor/chunk_manager.go</code>	Parallel upload dispatch, shard fetching, failure detection, and RS recovery calls.
<code>distributor</code>	<code>coordinator/distributor/health_monitor.go</code>	5-second recurring background ticker checking node latency, uptime, and disk usage.
<code>erasure</code>	<code>coordinator/erasure/encoder.go</code>	Reed-Solomon (3+1) Galois Field encoding and decoding routines via <code>klauspost/reedsolomon</code> .
<code>metadata</code>	<code>coordinator/metadata/store.go</code>	Thread-safe in-memory map storing file metadata, shard sizes, and checksums.
<code>grpcclient</code>	<code>coordinator/grpcclient/node_client.go</code>	Connection pooling, timeout wrapping, and error handling for storage node RPCs.

Storage Node Subsystem (C++17)

The storage nodes are zero-dependency C++17 micro-daemons compiled natively inside Ubuntu 22.04 containers.

Thread-Safe Storage Engine (`shard\_store.cpp`)

- All disk operations use `std::shared_mutex` for multi-reader single-writer locking.
- Files are stored directly on the volume filesystem: `/data/<file_id>_<shard_idx>.bin`.
- Atomic file stream writes with immediate `flush()` to prevent file corruption on power loss.
- Automatic storage accounting via `std::filesystem::space` and file size tracking.

gRPC Server Handler (`storage\_server.cpp`)

- Implements the Protobuf `StorageNode::Service` interface.
- Calculates SHA-256 over raw incoming shard buffers using OpenSSL EVP interfaces.
- Reports instantaneous latency, node uptime in seconds, shard counts, and available disk capacity.
- Non-blocking, multi-threaded request processing via gRPC worker thread pools.

5. Inter-Service Protocol & API Reference

gRPC Service Contract (`proto/storage.proto`)

The core storage contract is defined in standard Protocol Buffers v3 and compiled to both Go stubs and C++ runtime classes:

```
syntax = "proto3";
package dfs;
option go_package = "dfs/proto";

service StorageNode {
  rpc StoreShard      (StoreShardRequest)      returns (StoreShardResponse);
  rpc RetrieveShard   (RetrieveShardRequest)    returns (RetrieveShardResponse);
  rpc DeleteShard     (DeleteShardRequest)     returns (DeleteShardResponse);
  rpc HealthCheck     (HealthRequest)          returns (HealthResponse);
  rpc GetStats        (StatsRequest)           returns (StatsResponse);
}

message StoreShardRequest {
  string file_id      = 1;
  int32  shard_index  = 2;
  bytes  data          = 3;
  string checksum     = 4; // SHA-256 for transmission integrity
}

message StoreShardResponse {
  bool    success = 1;
  string  message = 2;
}

message RetrieveShardRequest {
  string file_id      = 1;
  int32  shard_index  = 2;
}

message RetrieveShardResponse {
  bool    success = 1;
  bytes   data    = 2;
  string  checksum = 3;
  string  message = 4;
}
```

Coordinator HTTP REST API

HTTP METHOD	ROUTE PATH	REQUEST BODY / PARAMS	RESPONSE SUMMARY
GET	/api/health	None	<code>{"status": "ok", "service": "dfs-coordinator"}</code>
GET	/api/cluster/status	None	All 4 nodes' health status, storage usage, latency, and reconstruction capability.
POST	/api/upload	Multipart Form: <code>file</code>	File ID, original size, shard sizes, upload duration, nodes used.
GET	/api/download/{id}	File ID in URL path	Binary file stream with <code>Content-Disposition: attachment</code> .
GET	/api/files	None	List of all tracked files, creation timestamps, and shard layouts.
DELETE	/api/files/{id}	File ID in URL path	Purges shard files from all 4 storage nodes and deletes catalog record.

6. Fault Tolerance, Failure Scenarios & Verification

Cluster Failure Matrix

SCENARIO	ACTIVE NODES	CLUSTER STATE	READ AVAILABILITY	WRITE AVAILABILITY
Nominal	Node 0, 1, 2, 3 (4/4)	OPTIMAL	100% (Direct fast-path without parity decoding)	100% (All 4 shards stored)
Single Node Loss (Data Node 1)	Node 0, 2, 3 (3/4)	DEGRADED	100% Fully Recoverable (RS reconstruction via Shard 0 + 2 + Parity 3)	Available (Tolerates single-node write failure)
Single Node Loss (Parity Node 3)	Node 0, 1, 2 (3/4)	DEGRADED	100% Fully Recoverable (All 3 raw data shards present; no RS decode required)	Available
Double Node Failure	Any 2 nodes (2/4)	CRITICAL	Unrecoverable (Only 2 shards available, 3 required)	Blocked (Uploads aborted)

Automated End-to-End Verification

The platform includes an automated demonstration suite (`scripts/demo.sh`) verifying mathematical integrity under active failure injection:

```
# Execution Output from scripts/demo.sh:

=====
Step 1 – Coordinator Health:  PASS ({"status":"ok"})
=====
Step 2 – Cluster Status:      PASS (4 healthy nodes detected)
=====
Step 3 – Upload Test File:    PASS (Size: 524,335 bytes → 3 data + 1 parity)
=====
Step 4 – Download & Verify:    PASS (SHA-256 matches original file)
=====
Step 5 – Kill Node 1:         storage-node-1 stopped → Cluster Degraded
=====
Step 6 – Degraded Download:    Reed-Solomon RECONSTRUCTION SUCCESS!
                               SHA-256 matches original payload perfectly
=====
Step 7 – Restart Node 1:      storage-node-1 restarted → 4/4 Nodes Healthy
=====
```

7. Deployment, Configuration & Operations Guide

Environment & Configuration Parameters

The coordinator service reads its cluster topology directly from environment variables, enabling zero-code deployments across Docker, Kubernetes, or bare metal:

VARIABLE	DEFAULT	DESCRIPTION
PORT	8080	Coordinator REST API listening port.
NODE_0_ADDR	localhost:50051	Host and gRPC port for Storage Node 0.
NODE_1_ADDR	localhost:50052	Host and gRPC port for Storage Node 1.
NODE_2_ADDR	localhost:50053	Host and gRPC port for Storage Node 2.
NODE_3_ADDR	localhost:50054	Host and gRPC port for Storage Node 3 (Parity Node).

Docker Compose Quickstart

To bootstrap the complete cluster including all 4 C++ storage nodes, the Go coordinator, and the Nginx frontend dashboard:

```
# 1. Start all services in background
docker compose up --build -d

# 2. View running containers
docker compose ps

# 3. Access web dashboard
open http://localhost:3000

# 4. View real-time cluster logs
docker compose logs -f coordinator
```

Technology Choices & Architectural Rationale

DESIGN CHOICE	SELECTED TECHNOLOGY	ARCHITECTURAL RATIONALE
Coordinator Runtime	Go 1.21	Exceptional concurrency via goroutines, native channel pipelines, and ultra-fast gRPC client multiplexing.
Storage Daemon	C++17	Predictable low latency, zero garbage collection pauses, direct POSIX filesystem access, and fine-grained memory controls.
Erasure Coding	Reed-Solomon (3+1)	Optimal balance of durability and storage efficiency (33% parity overhead vs 200% for 3x replication).
Inter-Node RPC	gRPC / HTTP/2	High-throughput binary protocol, multiplexed TCP streams, and compile-time type safety via Protobuf.
Integrity Verification	SHA-256	Guarantees cryptographic detection of bit-rot, network transmission errors, and partial writes.

Operational Readiness Certified

This specification outlines the complete system design, math verification, API boundaries, and runtime operations for Distributed File Storage (DFS).